

Program for CAM-32:

```
import cv2
import numpy as np
from ultralytics import YOLO
import threading

# =====
# Load YOLOv8n model
# =====
model = YOLO("yolov8n.pt")

# =====
# ESP32-CAM URLs
# =====
urls = [
    "http://192.168.137.137:81/stream",
    "http://192.168.137.147:81/stream",
    "http://192.168.137.172:81/stream",
    "http://192.168.137.122:81/stream"
]

width, height = 320, 240

# =====
# Threaded Camera Class
# =====
class CamThread:
    def __init__(self, url, name):
        self.cap = cv2.VideoCapture(url)
        self.frame = np.zeros((height, width, 3), dtype=np.uint8)
        self.ret = False
        self.name = name
        self.running = True

        self.thread = threading.Thread(target=self.update, daemon=True)
        self.thread.start()

    def update(self):
        while self.running:
            ret, frame = self.cap.read()
            if ret:
                frame = cv2.resize(frame, (width, height))
```

```

        self.frame = frame
        self.ret = True
    else:
        self.ret = False

def read(self):
    return self.ret, self.frame.copy()

def stop(self):
    self.running = False
    self.cap.release()

# =====
# Initialize Cameras
# =====
cams = [CamThread(urls[i], f"CAM {i+1}") for i in range(4)]

prev_state = False # for single print

# =====
while True:
    frames = []
    person_detected_any = False

    for cam in cams:
        ret, frame = cam.read()

        if not ret:
            frame = np.zeros((height, width, 3), dtype=np.uint8)
            cv2.putText(frame, f"{cam.name} Offline", (40, 120),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
        else:
            # =====
            # YOLO Detection
            # =====
            results = model(frame, verbose=False)

            for r in results:
                for box in r.bboxes:
                    cls_id = int(box.cls[0])
                    conf = float(box.conf[0])

                    if cls_id == 0 and conf > 0.5:

```

```

        person_detected_any = True

        x1, y1, x2, y2 = map(int, box.xyxy[0])

        cv2.rectangle(frame, (x1, y1), (x2, y2),
                      (0, 255, 0), 2)
        cv2.putText(frame, f"Person {conf:.2f}",
                    (x1, y1 - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6,
                    (0, 255, 0), 2)

    cv2.putText(frame, cam.name, (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)

    frames.append(frame)

# Print only once
if person_detected_any and not prev_state:
    print("Person Detected")

prev_state = person_detected_any

# Combine grid
top = np.hstack((frames[0], frames[1]))
bottom = np.hstack((frames[2], frames[3]))
combined = np.vstack((top, bottom))

cv2.imshow("YOLOv8 Multi-Cam (Threaded)", combined)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# =====
# Cleanup
# =====
for cam in cams:
    cam.stop()

cv2.destroyAllWindows()

```